

# Gerenciamento Distribuído de Frotas de Drones

Luan Rodrigues Araújo<sup>1</sup>

<sup>1</sup>Universidade Estadual de Feira de Santana - UEFS

Av. Transnordestina, S/N, Novo Horizonte, Feira de Santana, Bahia, Brasil, CEP 44.036-900

luanlra002@gmail.com

**Abstract.** *This work proposes a distributed system, implemented in the Go programming language and deployed via Docker containers, for coordinating a drone fleet in a military surveillance scenario. The architecture employs a Token Ring topology among four broker nodes, each responsible for a geographic sector, to achieve mutual exclusion during mission assignment without a central coordinator Tanenbaum and van Steen (2017); Coulouris et al. (2011). Sensors generate alerts that are forwarded to brokers, which, upon acquiring the token, select available drones and dispatch them via direct TCP connections Almeida (2020); Ribeiro (2016). State replication through periodic broadcast ensures fault tolerance: when a broker fails, its successor regenerates the token from the last known state and assumes control of the ring. The results demonstrate the viability and resilience of the proposed approach in organizing and automating the mission assignment process in distributed, failure-prone environments.*

**Resumo.** *Este trabalho propõe um sistema distribuído, implementado na linguagem de programação Go e implantado via contêineres Docker Vitalino and Castro (2021), para coordenação de uma frota de drones em um cenário de vigilância militar. A arquitetura emprega uma topologia de Token Ring entre quatro nós broker, cada um responsável por um setor geográfico, para alcançar exclusão mútua durante a atribuição de missões sem um coordenador central Tanenbaum and van Steen (2017); Ricart and Agrawala (1981). Sensores geram alertas que são encaminhados aos brokers, os quais, ao adquirirem o token, selecionam drones disponíveis e os despacham via conexões TCP diretas Donovan and Kernighan (2016). A replicação de estado por broadcast periódico garante tolerância a falhas: quando um broker falha, seu sucessor regenera o token a partir do último estado conhecido e assume o controle do anel. Os resultados demonstram a viabilidade e a resiliência da abordagem proposta na organização e automação do processo de atribuição de missões em ambientes distribuídos sujeitos a falhas.*

## 1. Introdução

O crescimento contínuo de cenários de vigilância e resposta rápida a eventos críticos tem evidenciado a necessidade de sistemas distribuídos capazes de coordenar frotas de agentes autônomos de forma resiliente, escalável e sem pontos únicos de falha Tanenbaum and van Steen (2017). Em contextos militares ou de segurança nacional, a indisponibilidade momentânea de qualquer nó coordenador pode comprometer missões em andamento e colocar em risco a integridade de toda a operação.

Diante desse cenário, este projeto propõe o desenvolvimento de um sistema distribuído de gerenciamento de frotas de drones, estruturado em quatro setores geográficos interdependentes (Norte, Sul, Leste e Oeste), cada qual dotado de sensores de monitoramento, drones de resposta e um broker coordenador. O objetivo central é garantir que, mesmo diante de falhas de nós, o sistema permaneça operacional, preservando a consistência do estado global da frota e a continuidade das missões em curso Coulouris et al. (2011).

A solução foi implementada utilizando sockets nativos TCP/IP na linguagem Go Donovan and Kernighan (2016), com comunicação entre processos baseada em troca de mensagens JSON codificadas sobre conexões TCP Almeida (2020). O ambiente de testes foi completamente dockerizado Vitalino and Castro (2021), permitindo a simulação de múltiplas instâncias de brokers, sensores e drones em um ambiente controlado e reproduzível. O mecanismo central de coordenação adota o algoritmo de Token Ring para exclusão mútua distribuída Ricart and Agrawala (1981), complementado por um mecanismo de broadcast para replicação de estado entre os nós do anel.

Este relatório discute o cenário do problema, os conceitos teóricos que servem de base para as escolhas de arquitetura, o projeto real dos componentes individuais do sistema e os testes e os resultados de desempenho correspondentes em termos de resiliência e correção do processo de atribuição de missões.

## **2. Fundamentação Teórica**

Esta seção detalha os pilares teóricos e as ferramentas tecnológicas que sustentam a solução para o problema de integração de rede e gerenciamento de dispositivos.

### **2.1. Comunicação em Redes com Sockets TCP**

A comunicação entre processos em redes de computadores pode ser realizada por meio de diferentes protocolos, sendo o TCP (*Transmission Control Protocol*) um dos mais utilizados por oferecer uma conexão orientada a fluxo, confiável e com controle de erro Ribeiro (2016). A interface de programação de sockets, permite a implementação direta da comunicação entre processos por meio de canais definidos por pares de endpoints compostos por endereço IP e porta Almeida (2020).

No contexto deste projeto, a utilização de sockets TCP é fundamental para estabelecer a comunicação entre todos os módulos do sistema: sensores, brokers e drones, conforme exigido pelas restrições do problema. A escolha pelo TCP justifica-se pela necessidade de garantir a entrega correta e ordenada das mensagens, especialmente para operações críticas como despacho de drones, passagem do token entre brokers e replicação de estado global Coulouris et al. (2011).

### **2.2. Algoritmo de Token Ring**

O Token Ring é um algoritmo clássico de exclusão mútua distribuída no qual um objeto lógico denominado token circula continuamente entre os processos participantes, organizados em uma topologia de anel Tanenbaum and van Steen (2017). No contexto deste projeto, apenas o processo que detém o token tem permissão para acessar os recursos compartilhados, como a fila global de alertas e o mapa de atribuição de drones. Isso significa que o token atua como uma espécie de chave que permite ao processo acessar

esses recursos importantes. A fila global de alertas e o mapa de atribuição de drones são fundamentais para o funcionamento do projeto, e o token garante que apenas o processo autorizado possa interagir com eles. Ricart and Agrawala (1981).

A principal vantagem do Token Ring é a ausência de um coordenador central: o próprio token carrega o estado compartilhado e o transfere de nó em nó, garantindo que somente um broker por vez realize atribuições de missões Tanenbaum and van Steen (2017). Quando um nó falha, mecanismos de detecção de timeout e de responsabilidade topológica permitem que o nó sucessor regenere o token e assuma o controle do anel, preservando a operação contínua do sistema Coulouris et al. (2011).

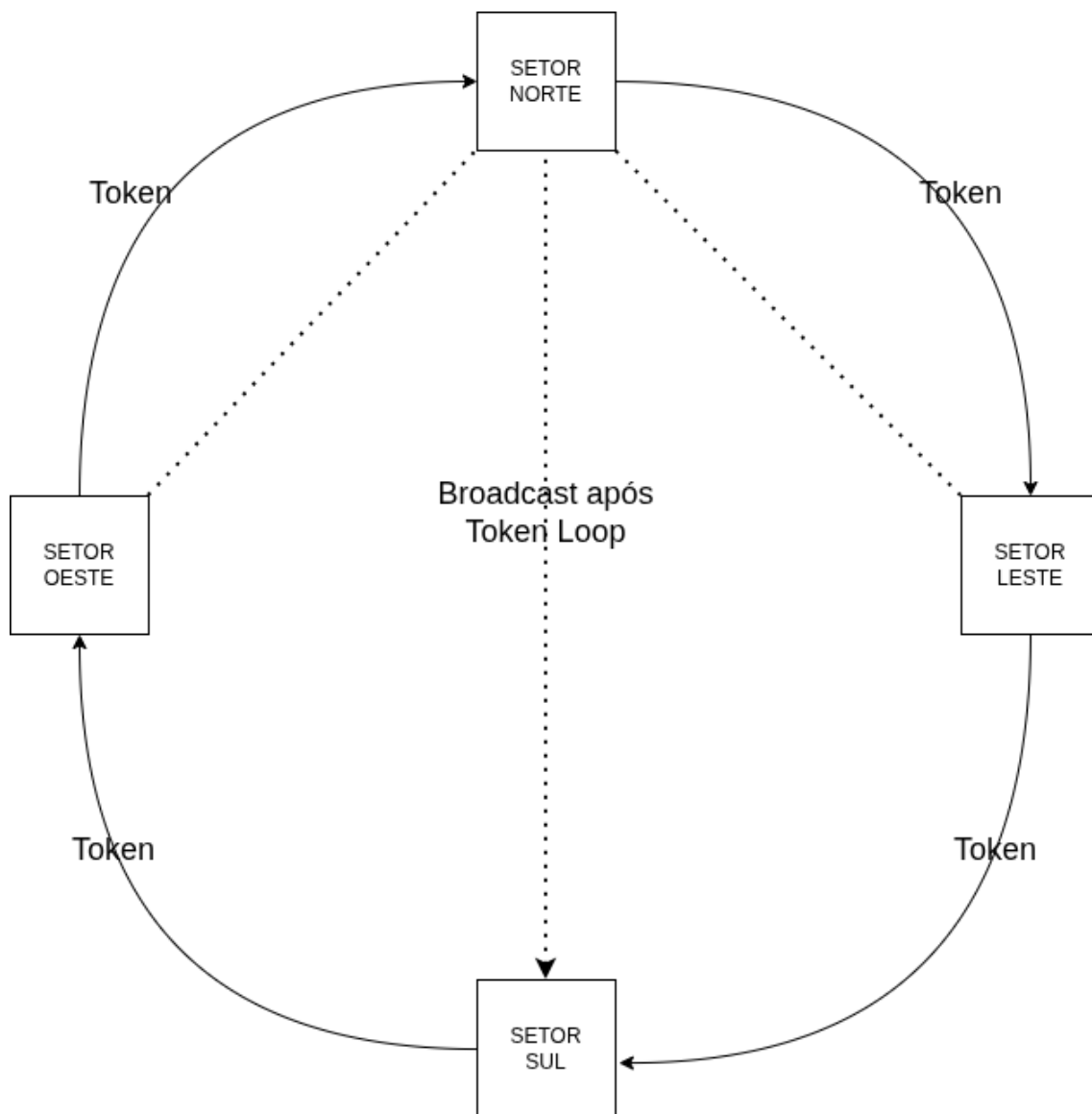


Figure 1. Token Ring

### 2.3. Replicação de Estado e Broadcast

Em sistemas distribuídos tolerantes a falhas, a replicação de estado é o mecanismo pelo qual cópias do estado global são mantidas em múltiplos nós, garantindo que a perda de

qualquer nó individual não resulte em perda de informação crítica Coulouris et al. (2011). Neste projeto, após cada ciclo de processamento do token, o broker que detém o token transmite uma cópia completa do estado global, que é chamada de StateUpdate, para todos os peers que estão ativos no anel. Isso é feito por meio de conexões TCP ponto-a-ponto em modo de broadcast Tanenbaum and van Steen (2017).

Cada broker receptor armazena essa cópia em uma estrutura de backup local. Quando o mecanismo de regeneração de token é ativado, o nó que assume o controle do anel restaura o estado mais recente a partir desse backup, garantindo que nenhum alerta pendente, missão em andamento ou histórico de ocorrências seja perdido em decorrência de falhas Coulouris et al. (2011).

## 2.4. Contêineres Docker

O Docker é uma plataforma de containerização que permite empacotar aplicações e suas dependências em unidades isoladas e reprodutíveis denominadas contêineres Vitalino and Castro (2021). Cada contêiner compartilha o kernel do sistema operacional hospedeiro, mas opera em um espaço de nomes isolado, o que garante consistência entre os ambientes de desenvolvimento, teste e produção.

Neste projeto, cada componente do sistema (broker, sensor e drone) é empacotado e executado como um contêiner Docker independente. O arquivo `docker-compose.yaml` orquestra a criação simultânea de dezoito contêineres (quatro brokers, oito sensores e oito drones), todos configurados exclusivamente por variáveis de ambiente, eliminando qualquer acoplamento de configuração em tempo de compilação Vitalino and Castro (2021).

## 2.5. Sistemas de Coordenação Distribuída de Agentes

Sistemas de coordenação de agentes autônomos em ambientes distribuídos precisam garantir a alocação eficiente de recursos compartilhados, como os drones disponíveis, entre demandas concorrentes provenientes de diferentes origens. A abordagem adotada neste projeto combina um mecanismo de exclusão mútua (Token Ring) com uma política de seleção de agentes baseada em afinidade geográfica e disponibilidade declarada, evitando conflitos de atribuição e maximizando a eficiência operacional da frota.

# 3. Metodologia

## 3.1. Proposta de Solução

A proposta deste projeto é o desenvolvimento de um sistema distribuído de gerenciamento de frotas de drones para vigilância em um cenário de setores geográficos interdependentes. O sistema visa atender aos principais desafios operacionais em ambientes hostis e sujeitos a falhas: coordenação sem ponto único de falha, preservação do estado global diante da queda de nós e resposta prioritária a alertas críticos. A solução é composta por três entidades principais:

- **Broker:** coordenador setorial responsável pela gestão do token, processamento de alertas e despacho de drones;
- **Sensor:** gerador de alertas baseado em leituras simuladas de grandezas físicas;
- **Drone:** agente executor de missões, com comunicação bidirecional com o broker.

Cada entidade foi implementada como um contêiner Docker separado e utiliza comunicação via sockets TCP, com mensagens serializadas em formato JSON. As conexões são estabelecidas e encerradas por demanda (modelo de conexão curta), garantindo simplicidade de implementação e ausência de state em camada de transporte.

### 3.1.1. Arquitetura Geral

A arquitetura do sistema pode ser dividida em dois planos distintos: o plano de dados, responsável pelo tráfego de alertas entre sensores e brokers e pelo despacho de ordens aos drones; e o plano de controle, responsável pela circulação do token entre os brokers do anel e pela replicação periódica do estado global por broadcast.

Os quatro brokers — Norte (172.16.201.7), Sul (172.16.201.8), Leste (172.16.201.11) e Oeste (172.16.201.12) — formam um anel lógico ordenado, todos escutando na porta 5000. Sensores e drones de cada setor conhecem a lista completa de brokers e implementam failover linear: na impossibilidade de alcançar o broker primário, tentam os demais em sequência. Cada drone escuta em uma porta TCP dedicada (6000 a 6007) para receber ordens de despacho diretamente do broker detentor do token.

### 3.1.2. Componentes do Sistema

**a) Broker** O broker é o componente mais complexo do sistema. Cada instância gerencia concorrentemente as seguintes responsabilidades:

- Recepção e enfileiramento local de alertas provenientes de sensores;
- Registro, atualização e monitoramento de heartbeat de drones;
- Participação no Token Ring: recepção, processamento e passagem do token;
- Replicação de estado por broadcast após cada ciclo de processamento Coulouris et al. (2011);
- Detecção de falhas de peers via ping periódico e bypass automático no anel Tanenbaum and van Steen (2017);
- Regeneração autônoma do token em caso de timeout detectado por topologia;
- Despacho TCP direto de ordens de missão ao drone selecionado Almeida (2020);
- Manutenção de log histórico completo do ciclo de vida de cada missão.

Internamente, o broker armazena os estados locais dos drones em uma estrutura `map[string]*DroneState`, protegida por mutex Donovan and Kernighan (2016). O token em circulação transporta o estado global compartilhado: a fila de alertas pendentes (`PendingAlerts`), o mapa de atribuições em andamento (`Assigned`), o mapa de status global dos drones (`DroneStatus`), o contador de rodadas (`Round`) e o log histórico de missões (`MissionLog`).

**b) Sensor** Cada sensor é executado como um processo independente que simula a geração de eventos de monitoramento. O componente gera periodicamente um valor numérico aleatório no intervalo  $[0, 100]$  com intervalo variável entre 5 e 15 segundos. Apenas valores superiores a 70 são classificados como alertas relevantes e encaminhados ao sistema, evitando sobrecarga desnecessária da fila global.

A prioridade de cada alerta é calculada no momento da emissão com base no valor medido: valores acima de 95 recebem prioridade 3 (crítico), valores entre 85 e 95 recebem prioridade 2 (alto) e valores entre 70 e 85 recebem prioridade 1 (médio). O sensor implementa failover linear na lista de brokers, tentando cada endereço em sequência até obter sucesso no envio.

**c) Drone** O drone é um agente executor que mantém um servidor TCP em porta dedicada para receber ordens de despacho do broker Ribeiro (2016). Ao iniciar, registra-se em todos os brokers da lista simultaneamente (`DRONE_REGISTER`), informando seu endereço de escuta. Depois disso, o sistema envia sinais de vida regulares a cada 10 segundos para todos os brokers ativos. Isso permite que qualquer nó que receba o token localize o drone e conheça seu status atual.

Ao receber um `DRONE_DISPATCH`, o drone verifica se a ordem é endereçada a si mesmo e, se estiver disponível, atualiza seu estado interno para `IN_MISSION`. Após a conclusão simulada da missão (com duração aleatória entre 10 e 20 segundos), reporta a conclusão via `DRONE_DONE` ao primeiro broker acessível da lista Donovan and Kernighan (2016).

### 3.1.3. Lógica de Comunicação e Protocolo de Mensagens

Todas as mensagens trocadas entre os componentes seguem uma estrutura JSON uniforme composta por dois campos: `type`, que identifica o tipo da mensagem, e `payload`, que carrega os dados específicos de cada tipo. O sistema define os seguintes tipos de mensagem:

### 3.1.4. Justificativas Técnicas

A escolha por Go (Golang) deve-se à sua leveza, ao excelente suporte a concorrência por meio de goroutines e canais, e à facilidade de trabalhar com sockets nativos sem dependências externas. O modelo de concorrência de Go é especialmente bom para lidar com muitas conexões ao mesmo tempo. Ele permite que cada conexão TCP seja tratada em sua própria goroutine. Isso significa que não há o peso de threads do sistema operacional, o que possibilita ter muitas conexões abertas simultaneamente sem perder desempenho.

A serialização de mensagens em JSON oferece legibilidade, extensibilidade e interoperabilidade sem a necessidade de esquemas externos ou compiladores de interfaces. O modelo de conexão curta (uma conexão TCP por mensagem) simplifica o gerenciamento de estado de rede e elimina a necessidade de detecção de half-open connections.

O uso de Docker e docker-compose permite simular toda a topologia distribuída em uma ou múltiplas máquinas físicas com configuração mínima, garantindo reprodutibilidade dos experimentos e facilitando futura implantação em infraestrutura real.

| Tipo             | Descrição                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------|
| ALERT            | Sensor → Broker: notificação de evento com ID, setor, tipo, valor e prioridade.                    |
| DRONE_REGISTER   | Drone → Todos os Brokers: registro inicial com endereço de escuta TCP.                             |
| DRONE_HEARTBEAT  | Drone → Todos os Brokers: sinal de vida periódico com status e missão atual.                       |
| DRONE_DONE       | Drone → Broker: notificação de conclusão de missão.                                                |
| DRONE_DISPATCH   | Broker → Drone: ordem de missão com ID, setor, tipo e prioridade do alerta.                        |
| TOKEN            | Broker → Broker: passagem do token com estado global completo da frota Ricart and Agrawala (1981). |
| STATE_UPDATE     | Broker → Peers: broadcast de replicação do estado global após cada rodada Coulouris et al. (2011). |
| PEER_PING        | Broker → Broker: sinal de vida periódico para manutenção do mapa <code>peerAlive</code> .          |
| Cobertura_SECTOR | Broker → Peers: aviso de cobertura de setor após queda de um broker.                               |

**Table 1. Tipos de mensagem do protocolo de comunicação**

### 3.2. Implementação

A implementação do sistema é composta por três arquivos Go principais — `broker.go`, `sensor.go` e `drone.go` —, além de um Dockerfile multistágio que compila e empacota cada binário de forma independente Vitalino and Castro (2021), e um `docker-compose.yaml` que orquestra a criação e configuração de todos os dezoito contêineres do sistema.

#### 3.2.1. Estrutura do Broker

O broker é estruturado em torno da `struct Broker`, que centraliza todo o estado local e os mecanismos de sincronização necessários para sua operação concorrente Donovan and Kernighan (2016). Os campos mais relevantes incluem:

- `drones (map[string]*DroneState)`: estado local dos drones registrados neste broker, protegido por `dronesMu`;
- `localAlerts ([]AlertPayload)`: fila de alertas recebidos localmente antes de deter o token, protegida por `localAlertsMu`;
- `orphanAlerts ([]AlertPayload)`: alertas de missões cujo drone foi declarado falho e devem ser redistribuídos;
- `tokenData (TokenPayload)`: última versão conhecida do estado global do token Ricart and Agrawala (1981);
- `backupTokenData (TokenPayload)`: backup do estado global recebido via broadcast de peers Coulouris et al. (2011);

- `peerAlive` (`map[string]bool`): mapa de liveness dos brokers do anel, atualizado pelo loop de ping Tanenbaum and van Steen (2017);
- `alertReady` (`chan struct`): canal de sinalização para acordar o `tokenLoop` quando novos alertas chegam Donovan and Kernighan (2016).

O método `main` inicializa o broker a partir de variáveis de ambiente (`SECTOR_NAME`, `PORT`, `MY_IP`, `RING_ADDRS`), lança as goroutines de monitoramento e inicia o listener TCP principal. O primeiro nó do anel (`ring[0]`) recebe o token inicial, garantindo que a circulação comece de forma determinística Ricart and Agrawala (1981).

### 3.2.2. Mecanismo de Token Ring

O algoritmo de Token Ring é implementado por um conjunto de métodos inter-relacionados Ricart and Agrawala (1981); Tanenbaum and van Steen (2017). Ao receber uma mensagem do tipo `TOKEN`, o método `receberToken` verifica se o broker já possui o token (descartando duplicatas que podem surgir durante recuperação de falhas), mescla o `DroneStatus` recebido com o estado local, combina os `MissionLogs` para preservar o histórico mais completo disponível, e lança uma goroutine dedicada `tokenLoop` para processar o token.

O método `tokenLoop` executa o seguinte ciclo de forma atômica e sequencial sob proteção do `tokenMu` Donovan and Kernighan (2016):

1. Absorve alertas órfãos de missões com drone falho, reinserindo-os na fila com prioridade máxima (3);
2. Absorve alertas locais recebidos desde a última passagem do token, inserindo-os na fila global;
3. Sincroniza o estado global dos drones com os estados locais conhecidos pelo broker;
4. Recupera missões de drones marcados como `FAILED` que ainda figuram no mapa `Assigned`;
5. Ordena a fila global por prioridade decrescente e, para prioridades iguais, por timestamp crescente Coulouris et al. (2011);
6. Para cada alerta pendente, seleciona um drone disponível com preferência geográfica ao setor do alerta, reserva atomicamente o drone no token e despacha a ordem via TCP direto ao drone Almeida (2020);
7. Limpa o mapa `Assigned` de entradas cujo drone já retornou ao status `AVAILABLE`;
8. Replica o estado atualizado para todos os peers vivos via `broadcastState` Coulouris et al. (2011);
9. Passa o token ao próximo nó vivo do anel via `passToken`, com bypass automático de nós mortos Ricart and Agrawala (1981).

A função `droneDisp` implementa a seleção de drone com dois estágios de preferência: primeiro, candidatos do mesmo setor do alerta; depois, candidatos de outros setores. Dentro de cada grupo, prioriza drones cujo endereço físico é conhecido pelo broker corrente, garantindo que a ordem de despacho possa ser entregue com sucesso Donovan and Kernighan (2016).



### 3.2.3. Mecanismo de Broadcast e Replicação de Estado

Após cada ciclo de processamento do `tokenLoop`, o método `broadcastState` envia uma cópia completa do `TokenPayload` atualizado para todos os peers marcados como vivos no mapa `peerAlive` Coulouris et al. (2011). Cada receptor, ao processar a mensagem `STATE_UPDATE`, verifica se o `Round` da atualização recebida é superior ao do backup armazenado e, em caso positivo, substitui o backup local.

Essa estratégia garante que todos os nós vivos do anel possuam, a qualquer momento, uma versão recente do estado global. Na ausência de falhas, o estado do backup nunca fica defasado por mais de um ciclo de `Token Ring`. Em caso de falha do detentor do token, o nó que assumir o controle do anel pode restaurar o estado a partir do backup, minimizando a perda de informação para o intervalo entre o último broadcast e o momento da falha Coulouris et al. (2011).

O método `mergeMissionLogs` é utilizado tanto durante a recepção do token quanto durante a regeneração para combinar dois logs históricos, preservando para cada `alert_id` a entrada com o status de ciclo de vida mais avançado, garantindo que informações sobre missões concluídas ou falhadas nunca sejam sobrescritas por entradas mais antigas Tanenbaum and van Steen (2017).

### 3.2.4. Detecção e Recuperação de Falhas

O sistema implementa três mecanismos independentes e complementares para detecção e recuperação de falhas:

Falhas de drones são detectadas pelo método `watchDroneHeartbeats`, executado a cada 10 segundos. Drones que não enviam `heartbeat` por mais de 30 segundos são declarados `FAILED`. Missões em andamento com esses drones são convertidas em alertas órfãos e reinseridas na fila global com prioridade máxima, garantindo que nenhuma missão seja abandonada definitivamente.

Falhas de brokers peers são detectadas pelo método `peerHeartbeats`, que envia mensagens `PEER_PING` a cada 8 segundos para cada endereço do anel. A falha de conexão atualiza o mapa `peerAlive` para `false`, o que habilita o `bypass` desse nó tanto na passagem do token (`passToken`) quanto no broadcast de estado (`broadcastState`). Quando um peer volta a responder, seu status é restaurado automaticamente.

A perda do token é detectada pelo método `tokenTimeoutGuard`, que monitora continuamente o tempo decorrido desde a última passagem do token por este broker Ricart and Agrawala (1981). A tolerância é calculada dinamicamente como (número de nós  $\times$  3 segundos + 5 segundos de margem). Ao detectar `timeout`, o broker utiliza a topologia do anel para determinar se é o responsável pela regeneração: se seu predecessor direto está morto, ele assume o controle do anel, restaura o estado a partir do backup e reinicia o `tokenLoop`.

### 3.2.5. Tratamento de Concorrência

O broker emprega múltiplos mutexes independentes para proteger estruturas de dados concorrentemente acessadas por diferentes goroutines, evitando deadlocks pelo princípio de aquisição de locks em ordem consistente e com escopos mínimos Donovan and Kernighan (2016). O `tokenMu` protege o estado do token e o flag `hasToken`; o `dronesMu` protege o mapa de drones; o `localAlertsMu` protege a fila local de alertas; o `orphanMu` protege a fila de alertas órfãos; e o `peerMu` (`RWMutex`) protege o mapa `peerAlive`, permitindo leituras concorrentes durante a seleção de drones Donovan and Kernighan (2016).

O guard `tokenLoopRunning`, protegido pelo `tokenMu`, impede que múltiplas goroutines de `tokenLoop` sejam lançadas simultaneamente — cenário que poderia ocorrer durante a regeneração do token em ambientes com múltiplos nós tentando assumir o controle concorrentemente Coulouris et al. (2011). O canal `alertReady` com capacidade de 64 elementos permite que alertas cheguem em rajada sem bloquear a goroutine receptora, enquanto o `tokenLoop` drena o canal antes de iniciar o processamento para descartar notificações acumuladas de rodadas anteriores Donovan and Kernighan (2016).

### 3.3. Experimentos e Resultados

Para validar o funcionamento e a robustez do sistema desenvolvido, foram realizados experimentos que simulam cenários realistas de operação com múltiplos sensores, drones e brokers ativos simultaneamente. Os testes tiveram como objetivo avaliar o comportamento do sistema em situações de operação normal, de alta carga de alertas e de falhas de nós.

### 3.4. Ambiente de Testes

Os experimentos foram conduzidos utilizando o ambiente de containerização Docker com `docker-compose` Vitalino and Castro (2021), permitindo a simulação simultânea de dezoito contêineres: quatro brokers setoriais, oito sensores (dois por setor, um de tipo Radar e um de tipo Naval) e oito drones (dois por setor). Todos os contêineres operam em modo de rede `host` (`network_mode: host`), compartilhando a pilha de rede do hospedeiro, com endereços IP fixos por máquina para cada broker.

Essa abordagem viabiliza a criação de um ambiente controlado e reproduzível, permitindo a simulação de topologias distribuídas em uma única máquina de desenvolvimento ou em múltiplos hosts físicos sem alterações no código Vitalino and Castro (2021). A configuração por variáveis de ambiente garante que o mesmo binário compilado possa ser utilizado como qualquer broker, sensor ou drone, bastando ajustar os parâmetros de inicialização.

### 3.5. Simulação de Geração de Alertas e Priorização

O componente de sensor gera alertas com valores aleatórios distribuídos uniformemente. Apenas valores acima de 70, correspondentes a aproximadamente 30% das leituras resultam em alertas enviados ao sistema. Desses, aproximadamente 5% são classificados como críticos (prioridade 3), 15% como altos (prioridade 2) e 80% como médios (prioridade 1).

A fila global de alertas no token é ordenada por prioridade decrescente e, para prioridades iguais, por timestamp crescente (FIFO dentro do mesmo nível de prioridade) Coulouris et al. (2011). Isso garante que eventos críticos como detecções de atividade naval ou aérea de alta intensidade, sejam sempre atendidos antes de eventos de menor urgência, independentemente da ordem de chegada ao sistema.

### **3.6. Testes de Exclusão Mútua e Atribuição de Missões**

A garantia de exclusão mútua na atribuição de missões foi verificada pela análise dos logs de despacho Ricart and Agrawala (1981). A pré-reserva atômica do drone no token (atualização dos campos `Assigned` e `DroneStatus` antes do despacho TCP), garante que, mesmo que o token circule rapidamente, a atribuição seja detectável por todos os brokers subsequentes que receberem o token na mesma rodada Ricart and Agrawala (1981).

Em cenários de alta densidade de alertas, com múltiplos sensores gerando eventos concorrentemente, o sistema demonstrou capacidade de enfileirar os alertas localmente em cada broker e absorvê-los na fila global no momento em que o token passa, sem perda de eventos Donovan and Kernighan (2016). A estrutura de fila local independente por broker desacopla a recepção de alertas do ciclo de processamento do token, permitindo que o sistema processe rajadas de eventos sem degradação.

### **3.7. Testes de Tolerância a Falhas**

A tolerância a falhas foi avaliada por meio da interrupção forçada de contêineres de broker durante a operação normal do sistema. Quando o broker detentor do token é interrompido, o mecanismo de `tokenTimeoutGuard` detecta a ausência do token após a janela de tolerância calculada dinamicamente. O nó sucessor imediato, ao verificar que seu predecessor direto está morto e que nenhum outro predecessor vivo existe entre eles, assume a responsabilidade de regenerar o token Tanenbaum and van Steen (2017).

O estado restaurado a partir do `backupTokenData` recebido via broadcast pelo broker predecessor na última rodada completa antes da falha, permite que a operação continue com mínima perda de informação Coulouris et al. (2011). Missões que estavam em andamento no momento da falha do broker são preservadas no log histórico e, caso o drone associado também falhe, são redistribuídas como alertas órfãos na próxima rodada.

A falha simultânea de múltiplos brokers foi também contemplada pelo mecanismo de bypass em `passToken`: ao tentar passar o token, o broker percorre o anel em ordem, saltando todos os nós marcados como mortos, até encontrar um nó vivo. Se todos os outros nós estiverem mortos, o broker retém o token e continua operando como único coordenador ativo da frota Tanenbaum and van Steen (2017).

### **3.8. Avaliação Geral**

Os experimentos demonstraram que o sistema é capaz de lidar com múltiplos sensores gerando alertas concorrentemente, com atribuição correta de drones sem conflitos, e com a recuperação autônoma diante de falhas de brokers e drones Coulouris et al. (2011). A arquitetura baseada em Token Ring demonstrou-se adequada para garantir exclusão mútua sem coordenador central Ricart and Agrawala (1981), enquanto o mecanismo de broadcast de estado proveu a redundância necessária para a recuperação de falhas com mínima perda de dados Tanenbaum and van Steen (2017). O log histórico de missões,

integrado ao token e replicado via broadcast, proveu rastreabilidade completa do ciclo de vida de cada missão ao longo de toda a operação.

#### **4. Conclusão**

O projeto demonstra a viabilidade de um sistema distribuído de coordenação de frotas de drones baseado em Token Ring Ricart and Agrawala (1981) com replicação de estado por broadcast Coulouris et al. (2011), implementado integralmente com primitivas de rede nativas da linguagem Go Donovan and Kernighan (2016). A arquitetura adotada garante exclusão mútua na atribuição de missões, tolerância à queda de nós brokers e drones, e preservação do histórico completo de operações, sem a necessidade de um coordenador central ou de um banco de dados externo Tanenbaum and van Steen (2017).

A combinação do Token Ring para controle de acesso à fila global com o broadcast periódico de estado entre os brokers mostrou-se eficaz na prática: a janela de inconsistência após uma falha de broker é limitada ao intervalo entre o último broadcast bem-sucedido e o momento da falha, que no pior caso corresponde a uma fração de um ciclo de rotação do token Coulouris et al. (2011). A responsabilidade topológica para regeneração do token evita a duplicação do token em cenários de recuperação concorrente Ricart and Agrawala (1981).

Apesar da solidez da base implementada, o projeto ainda apresenta oportunidades relevantes de evolução. A janela de tolerância do `tokenTimeoutGuard` é calculada de forma conservadora e poderia ser ajustada dinamicamente com base na latência real observada entre os nós Coulouris et al. (2011). O mecanismo de broadcast não implementa confirmação de recebimento, o que pode resultar em perda de atualizações de estado em redes com alta taxa de perda de pacotes Tanenbaum and van Steen (2017). A preferência geográfica na seleção de drones poderia ser enriquecida com critérios adicionais, como carga de trabalho histórica ou distância estimada até o setor do alerta. Por fim, a interface de monitoramento do sistema poderia ser complementada por um painel visual que exibisse em tempo real o estado de cada broker, drone e fila de alertas Donovan and Kernighan (2016), facilitando a operação e a depuração do sistema em produção.

## References

- Almeida, E. (2020). Trabalhando com o protocolo TCP/IP usando go. iMasters. Acesso em: 10 abr. 2026.
- Coulouris, G., Dollimore, J., Kindberg, T., and Blair, G. (2011). *Distributed Systems: Concepts and Design*. Addison-Wesley, Boston, 5 edition.
- Donovan, A. A. A. and Kernighan, B. W. (2016). *A linguagem de programação Go*. Novatec Editora, São Paulo, 1 edition.
- Ribeiro, F. (2016). Sockets TCP na prática: comunicação cliente-servidor. Embarcados. Acesso em: 10 abr. 2026.
- Ricart, G. and Agrawala, A. K. (1981). An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM*, 24(1):9–17.
- Tanenbaum, A. S. and van Steen, M. (2017). *Distributed Systems: Principles and Paradigms*. CreateSpace, 3 edition.
- Vitalino, J. F. N. and Castro, M. A. N. (2021). *Descomplicando o Docker*. Novatec Editora, São Paulo.